Enigma OS 2.0

Description

Enigma OS is a fully-configurable in-world control surface for VRChat worlds. It pairs the look and feel of a hardware MIDI launchpad or DJ mixer with a flexible action system that lets you wire up nearly any in-world behaviour — material swaps, screen shader effects, skybox changes, AudioLink modulation, GameObject toggles, Udon variable writes, transform changes, teleports, and more — all without writing a single line of code. Configuration happens entirely through the Unity inspector: you build folders of buttons, drop actions onto each one from a categorized picker, and Enigma OS bakes everything down to runtime arrays consumed by a single Udon executor at play time.

Out of the box, Enigma OS ships with two ready-to-use prefabs (a button-only Launchpad and a fader-equipped Mixer with a built-in AudioLink controller and screen display), a library of templates covering common patterns (object toggles, material swaps, skybox switchers, world-stat displays, persistent presets, and turnkey folders for popular shader sets), and integrated support for popular third-party systems (OhGeezCmon Access Control, ProTV, Flatline). Advanced features include exclusivity tags, time-based auto-changing, conditional and stepped actions, color palettes, persistent per-user preset storage via VRChat PlayerData, dynamic faders that re-bind based on which buttons are active, and multi-controller support with room boundaries for spatially-separated installs. The result is a single asset that scales from a one-button toggle to a full mixer-style FX rig.

The Mixer prefab’s design is inspired by the Roland VR-50HD AV Mixer.

Installation and Dependencies

How to Install:

1. You must have AudioLink imported into your project. Open VRChat Creator Companion, select your project, and add AudioLink from the official packages.
2. Download the .unitypackage and import Enigma OS into your world project. You should see the files under the “Cozen” folder.
3. At the root of the Enigma OS folder are two prefabs, a Launchpad variant and a Mixer variant. Drag the one you prefer to use in your scene. Both prefabs use the same Enigma OS. The Launchpad variant has only buttons, while the Mixer adds faders and a screen.
4. If you do not have AutoLink installed, you will be prompted to import it. This is required for the Mixer. If you only plan to use the Launchpad, you can choose to ignore this message.

Supported Third Party Packages:

Shader Systems

- Mochie Screen FX:
    - Free version: https://github.com/MochiesCode/Mochies-Unity-Shaders
    - Extended Patreon version: https://www.patreon.com/c/mochieshaders/posts
    - There are templates included for both versions.
- June Shaders:
    - Purchase from: https://kleineluka.gumroad.com/l/june
    - Complete the full installation process as per June Shaders documentation
- Bean FX:
    - Download from: https://booth.pm/ja/items/8178293
- Taco FX:
    - Download from: https://booth.pm/ja/items/8043371

Access Control and Video Player

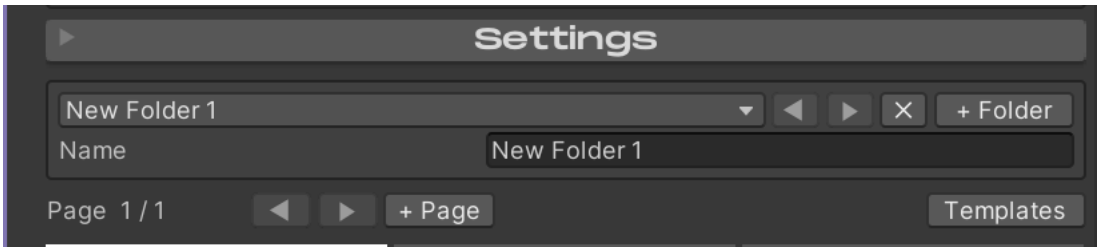
- OhGeezCmon Access Control: Runtime whitelist management
    - Repository: https://github.com/OhGeezCmon/VRC-AccessControl
    - Allows adding users to the whitelist during runtime
    - Install the package from the repository
- ProTV: Video player with whitelist support
    - Website: https://protv.dev/
    - Add the VCC repository and install from Creator Companion
- Flatline Open Decks Manager:
    - Download from: https://lavysworlds.gumroad.com/l/flatline

Using Enigma OS

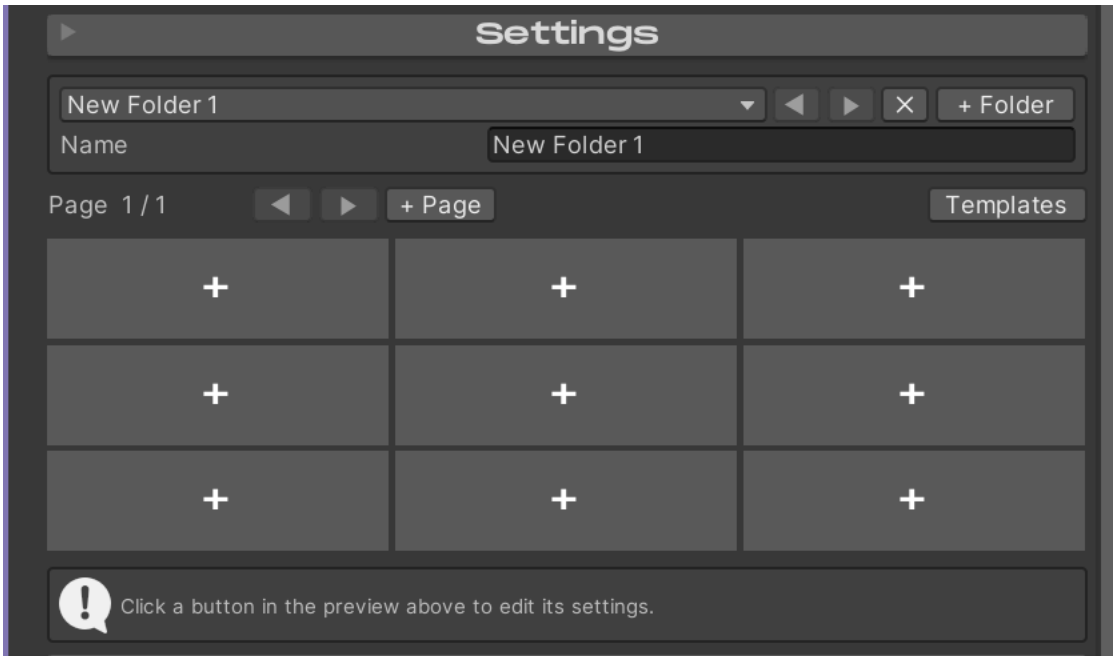
After importing and adding one of the prefabs to your scene, click on the prefab to start using Enigma OS.

This editor is where you will do all your configuration. To begin, click the “+ Folder” button.

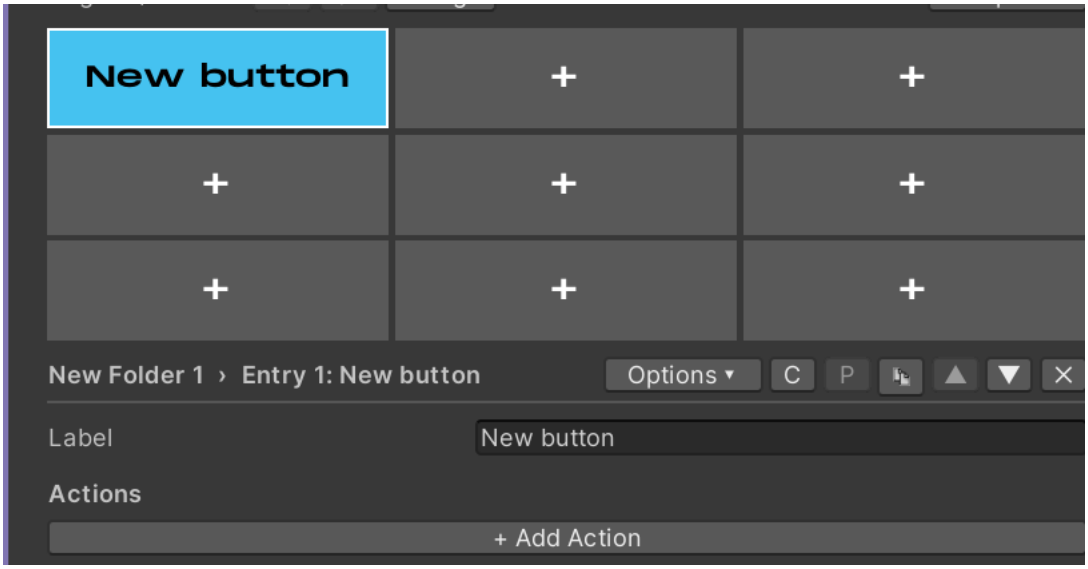
You will see a new folder populated. You can change the name to whatever you desire. You can make as many folders as you want. Use the dropdown to re-order the folders or select which folder to edit. You can also navigate folders using the arrow buttons. The X button deletes the currently selected folder.



You will also see the Button Grid appear. To edit a button in your folder, click on one of these slots. You can also add or navigate between pages using the arrow keys and “+ Page” button. The Templates button allows importing/exporting a folder, see [Template System]. Basic templates are included for many use cases and shader sets.



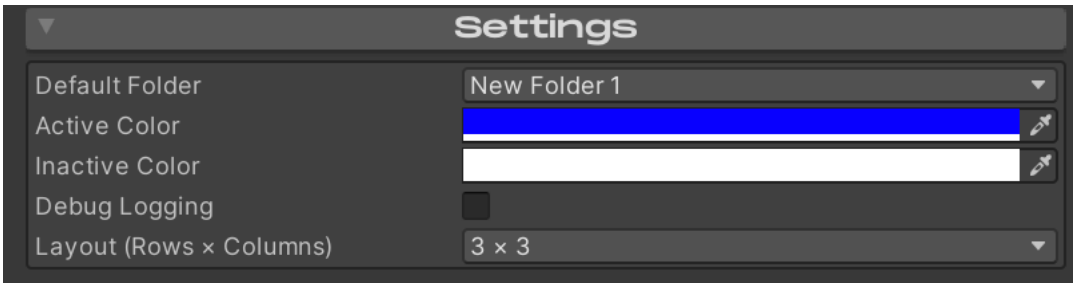
Doing so will make a new button, you can change the name to whatever you want.



To configure what the selected button does, add an action with “+ Add Action”. This will show a list of all available actions. For details on what each action does, see [Actions Overview].

Each button can also be configured with additional options using the “Options” dropdown. These are covered in [Advanced Features] and [Faders].

The C and P buttons allow copying and pasting buttons to a different button slot on your controller, even across different folders. The duplicate button makes a duplicate of the selected button on the next empty slot in the current folder. The arrow keys allow quickly switching to the next/previous button in the folder. The X button deletes the currently selected button.



At the top of Enigma OS is a Settings foldout. Here you can choose the default folder that is shown when your world is launched. “Active Color” and “Inactive Color” are the default colors used for the buttons in the “On” and “Off” state. Debug Logging should be left disabled unless you’re trying to make a bug report of any issue, it is very noisy. Layout controls the layout of the Button Grid. This shouldn’t be changed unless you’re making a custom controller, see [Making Custom Controllers].

The Faders foldout is where you can configure static faders. See [Using Faders].

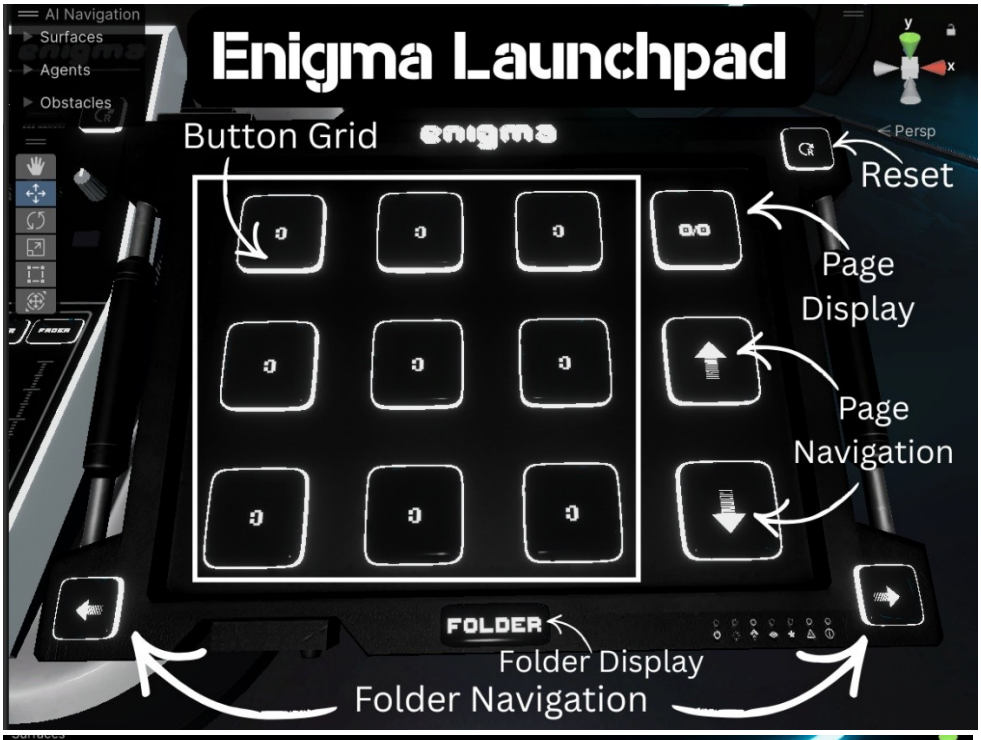
The Whitelist foldout is where you can configure access control. See [Whitelist (Access Control)].

The Hardware foldout is where you can add more buttons or faders. The prefabs are already wired, so this is only useful for [Making Custom Controllers] or re-assigning the references if you happen to “Reset” the entire editor script.

The Build button allows you to quickly set the scene to your configured default state. It’s not required to click this after every change, Enigma OS will call Build on play mode entry and when building your world through the VRChat SDK.

Lastly, you can quickly open these docs using the Documentation button or join the Enigma Discord for support or questions.

Using Prefabs



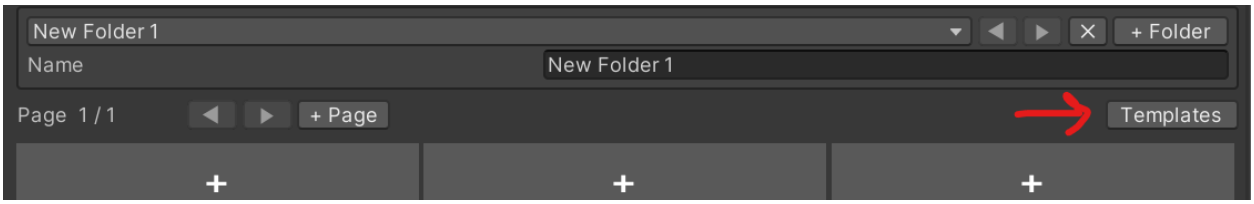
Both the Launchpad and Mixer run on Enigma OS. The only difference is the Mixer adds 9 customizable faders, a screen with a built-in AudioLink controller, screen texture, and waveform display, and additional buttons for fader control or hotkeys. The “Alternate Fader Control” button

switches between hand collider control and VRC pickup control, allowing manipulation of the faders in play mode or on Desktop.

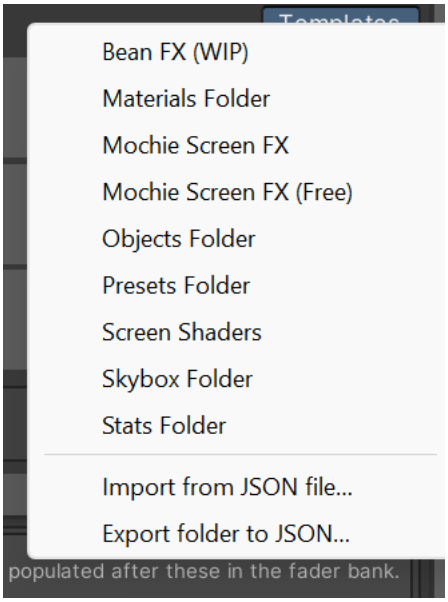
You can use more than one controller in the same scene, but you can only ever have one AudioLink controller! Make sure you delete any existing AudioLink controller, or remove the one under the “Screen” child object of the Mixer prefab.

Template System

Templates allow importing or exporting folders. Basic templates are included for various use cases and shader sets. To get started with templates, click the “Templates” button.



This opens a menu to choose which template to import, or to export the current folder as a template. Saving a template JSON to the “Templates” folder under the “Enigma OS” folder will add your new template to this list.



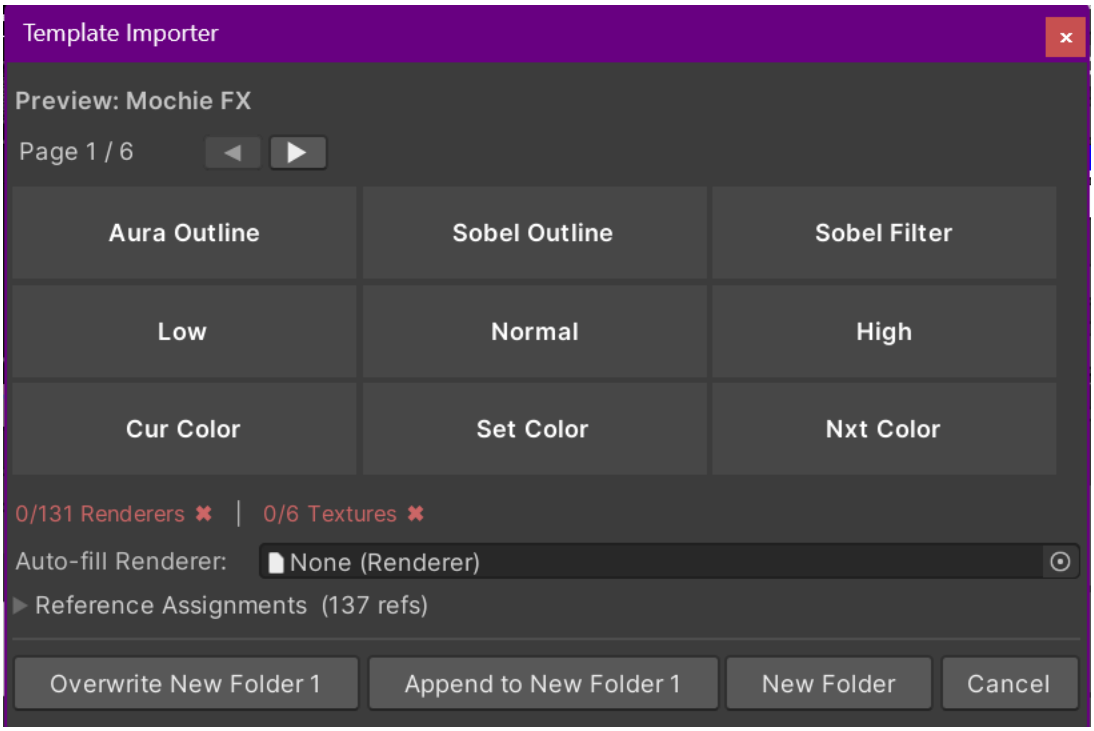
Basic templates include:

- Objects Folder: A page of object toggles, for toggling the active state of objects.
- Materials Folder: A page of material toggles, for swapping materials on a renderer.
- Skybox Folder: A page of skybox toggles with an auto-change button, for changing the skybox in your scene.
- Stats Folder: A page of stats displays, for viewing stats about your world. Can pull stats directly from the VRChat API like total visits.

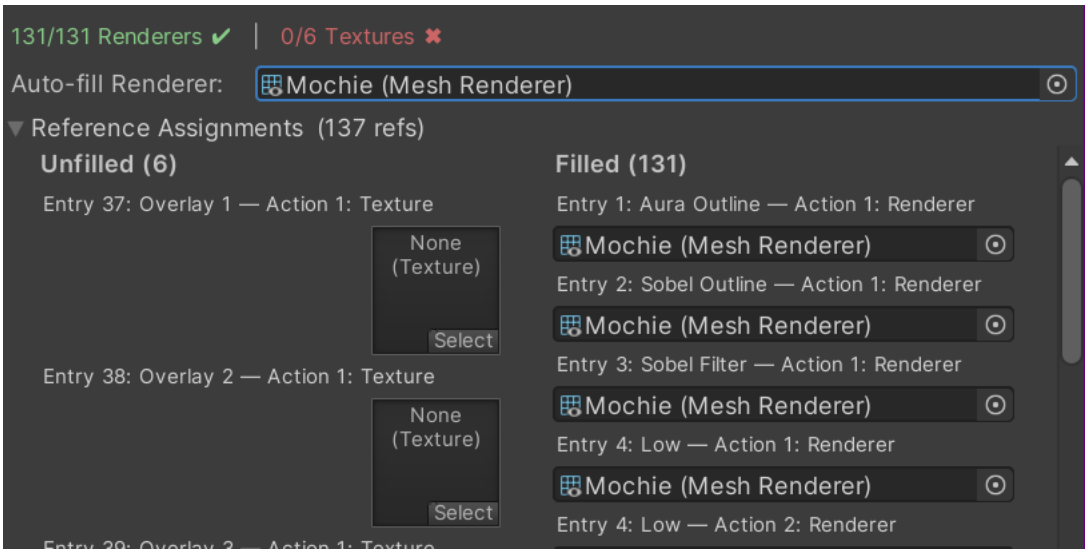
- Presets Folder: Allows saving the current state of the controller as a preset. Includes save, load and clear buttons, and 24 blank preset slots. Presets are persistent and save to the PlayerData, allowing each user to make their own and load them when using the controller. More on that in [Using Presets].
- Screen Shaders: This folder allows launching screen shaders configured on separate materials using templated renderers. See [Screen Shader Setup].
- Mochie, Bean FX, etc: These templates are specific to various shader sets. You must have these shaders imported to use them. See [Screen Shader Setup].

More templates may be added later, so this list might not be exhaustive.

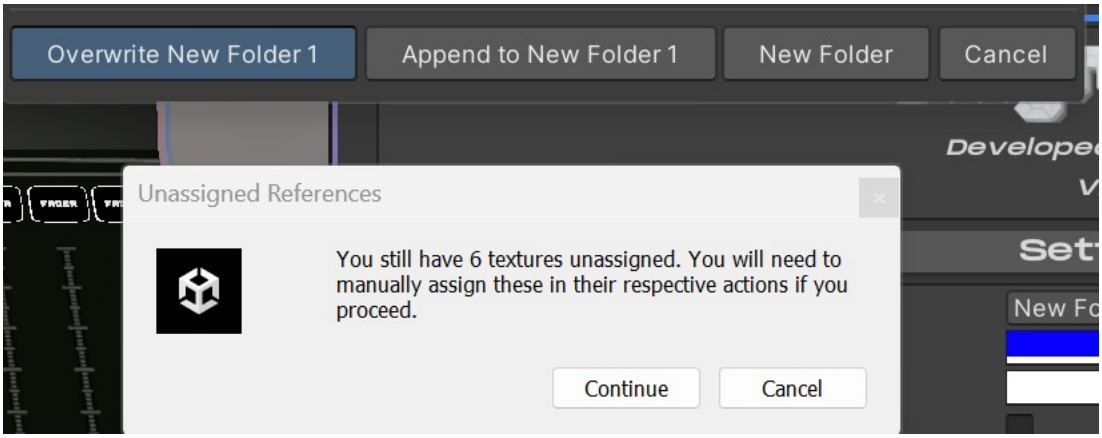
Clicking on a template in the menu will open the Template Importer. This is where you can preview the buttons to be imported, and assign references if required.



Use the arrow keys to navigate pages in the preview. The Reference Assignments foldout lets you assign the references like renderers, textures, etc. If you do not assign these, you will need to do so in the individual button actions.



Use the Auto-fill Renderer field to quickly assign all renderer fields with the chosen renderer. This is useful for shader set templates, where these are intended to use a single renderer. Filled references move to the right side, so you can see on the left what remains to be assigned.



When you are finished, click one of the buttons to import the template. Overwrite will replace the currently selected folder. Append will add the template buttons to the current folder, keeping any currently configured buttons. New folder will import it as a new folder. If you try to import with unassigned references, a warning will pop-up.

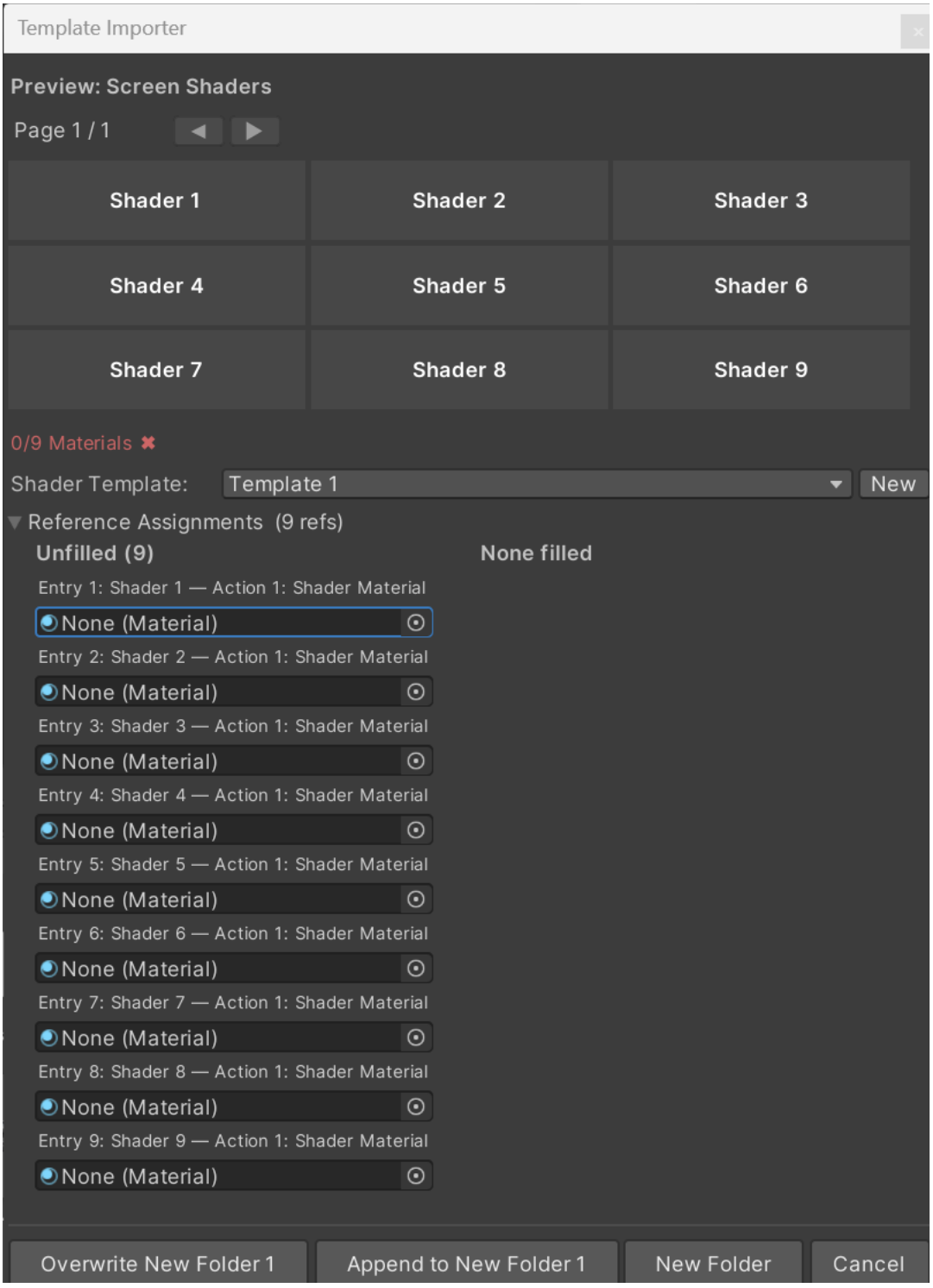
Note: Templates are great for learning how to make your own layouts! You can copy/paste buttons from a template into your own custom folder.

Screen Shader Setup

There are two ways to go about using screen shaders with Enigma OS, using one material/renderer or using multiple materials and a templated renderer.

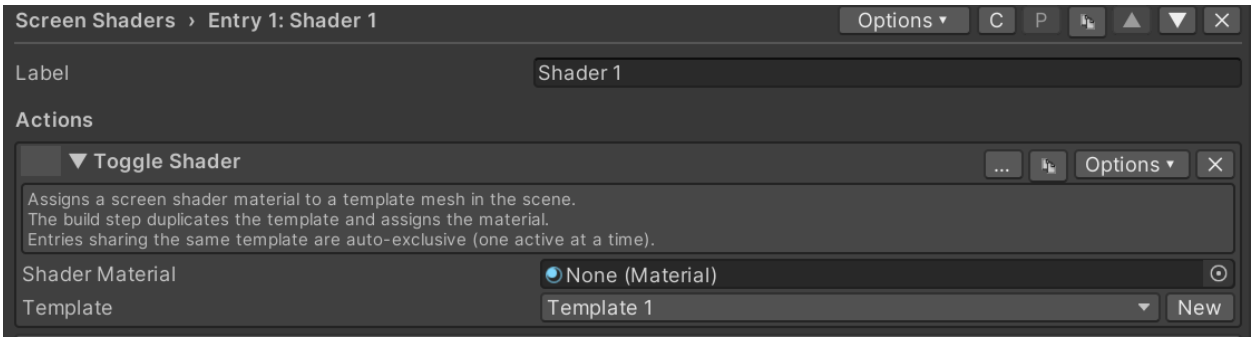
Multiple Material Setup

If you already have a collection of materials with separate effects configured on each material, you can use the multiple materials approach, which uses the “Toggle Shader” action. The easiest way to do this is to import the “Screen Shaders” template.



Assign each screen shader material in Reference Assignments. If you have more, import again and choose the Append option to add another page, or simply duplicate the button in the button

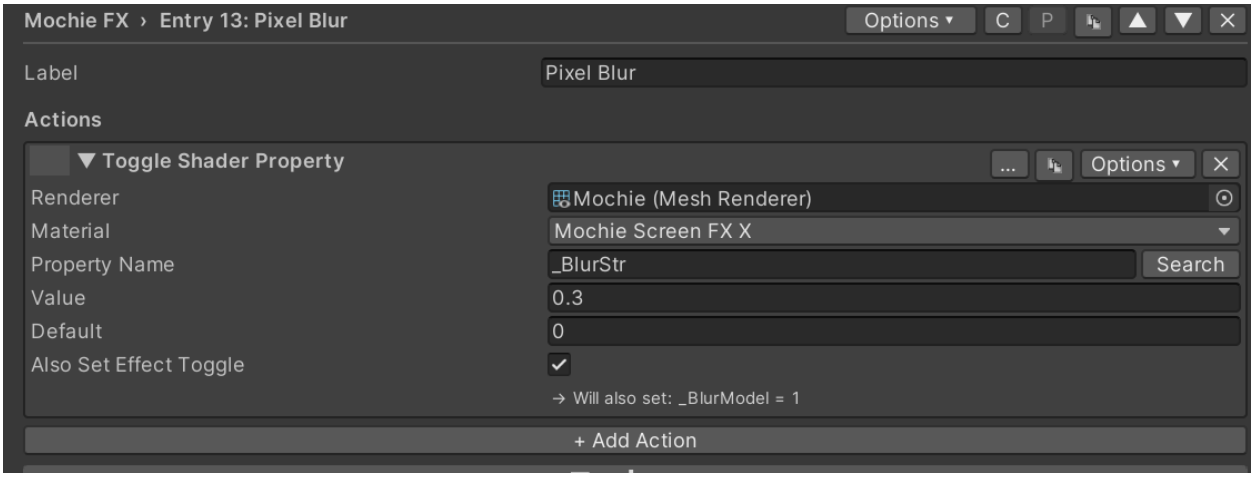
grid. Click the “New” button to add a shader renderer to the scene. This is an editor-only cube with no collider. Size the cube around the area you want effects to be visible. Enigma OS will handle duplicating this template for each material, and each button toggles the active state of the duplicated renderer. If you’re using more than one controller or have multiple rooms that you want separate effects in, you can make separate templates. Choose the right one in the template dropdown of the importer, or in the “Toggle Shader” action.



Single Material Setup

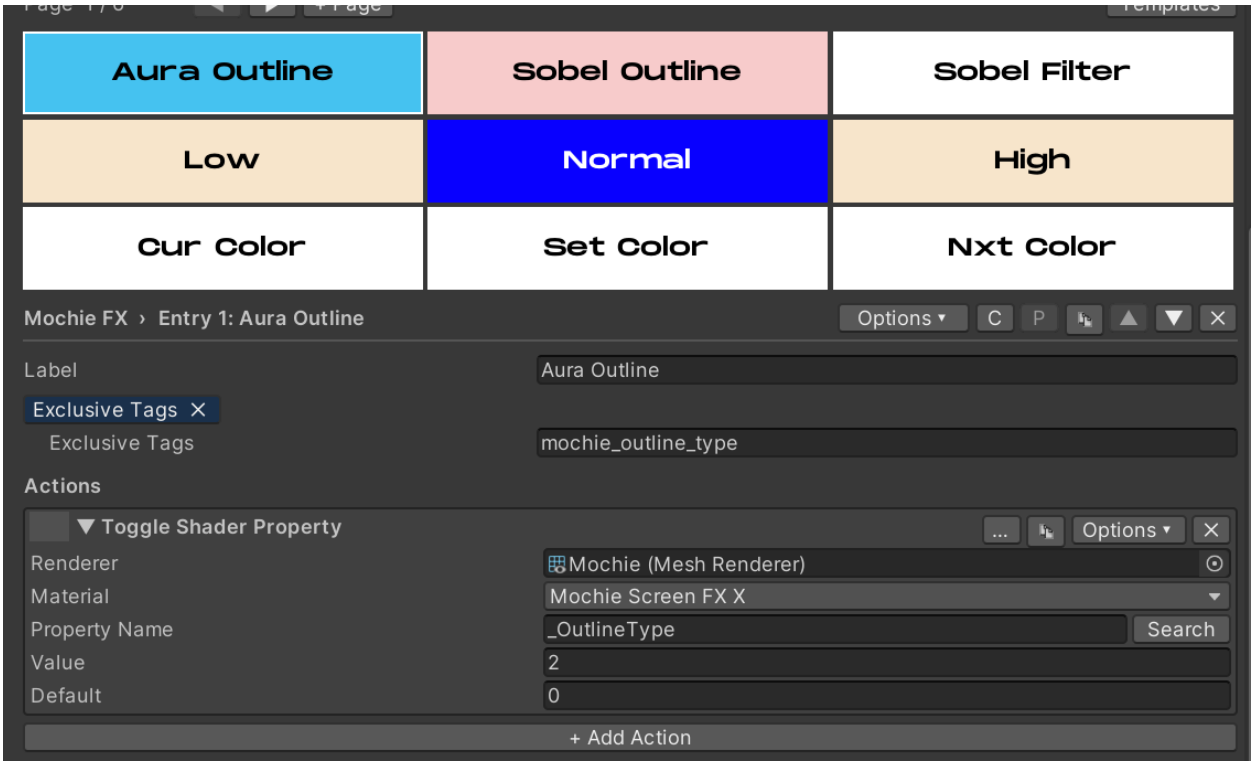
You can also control screen shaders using just a single material / renderer. This is the approach used by the shader set templates. For quick setup, import the template for the shader set you want to use. See [Template System]. The renderer you choose should be a cube (or sphere, but cube is recommended) and should be sized around the area you want the effects to render in. Remember to remove the default collider from this object. Assign a material using the chosen shader set to that renderer. You do not need to enable the effects manually within the material or lock the material! Enigma OS handles configuring the material and locking it based on what is configured in the Enigma OS editor.

To make your own folder using the single material approach, add a “Toggle Shader Property” action on any button, and assign your renderer. Then, choose which property you want the button to toggle.



The value field is the value that property will be set to when the button is toggled. The default field is the “off” state of the toggle, and will be set on scene start.

The “Also Set Effect Toggle” checkbox controls if the toggle also enables the effect toggle itself. For shaders like Mochie Screen FX, this is required! You should leave this checked unless you want separate buttons for activating the effect and for controlling an effect setting. An example of that is the outline buttons in the Mochie FX template. “Aura Outline” is set to the toggle property itself, and the low/medium/high buttons control the strength of the outline effects and have “Also Set Effect Toggle” disabled.



You can also customize your shader folder even more, with exclusivity, auto-changing, value displays, step buttons, color selectors, delays, trigger conditions, and custom button colors. See [Advanced Features] to learn how to use these, or take a look at the shader set templates for examples.

You can also have buttons assign faders to the fader bank when enabled, see [Using Faders].

Additional Notes:

You should place your templates/renderers under a parent object called “Shaders” or something similar, then place a user-facing toggle in your world to disable/enable that parent object locally. This allows players to turn on/off the screen effects for performance or sensitivity reasons.

Enigma OS officially supports the following screen shader sets:

- Mochie Screen FX (both free and paid versions)
- June FX (both free and paid versions)
- Bean FX
- Taco FX

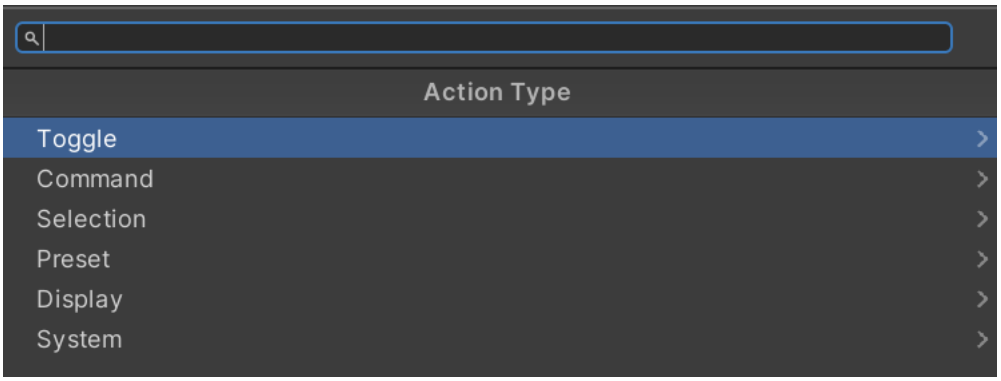
Other shader sets will work, but Enigma OS may silently skip locking or keyword management if the shader structure differs from the patterns established by these shader sets. If you find a shader set isn’t being locked properly (effects don’t render in the published world), manually lock the material before building. A “Toggle Shader Keyword” action is included to allow setting shader keywords manually (this isn’t required for the officially supported shader sets, Enigma OS manages keywords automatically for any shader using the standard [Toggle(KEYWORD)] attribute convention).

It’s recommended to use the single material approach for most setups. This is more optimized in most cases, but whether or not that is the case largely depends on the shader set and what effects you configure. If you find using a single material is too unoptimized or you plan on configuring a large number of effects using a shader set like June FX, the multiple material approach may be more optimized, as the benefits of using a single material/renderer become outweighed by a massive single shader compiled with many heavy effects. Test in game before publishing.

If certain effects do not render, check that a material/renderer not controlled by Enigma OS renders the effect. If not, the issue is not with Enigma OS but your configuration or scene setup. Some shader effects require Depth Lights, make sure you have one in your scene. You do NOT need to add multiple depth lights to your scene if using multiple shader sets, only one is required. They force Unity to populate _CameraDepthTexture, which is a global resource any depth-reading shader can sample.

Actions Overview

Buttons can be configured to do any number of actions. Toggle actions are stateful, with on/off states, while Command actions perform an action without tracking an on/off state. Selection actions allow making color selectors, and preset actions are for the presets system. Display actions display a value on the button, and System actions handle Enigma OS functions like navigation.



Toggle Actions:

Toggle Autochange Group: This toggles auto-changing on/off for the specified auto-changing group. See [Advanced Features].

Toggle Material: This swaps the material on the specified renderer. If multiple materials exist on the renderer, a material index field will be shown. The default field controls which material is applied on deactivation, and is auto-populated from the renderer's current material when the field is empty.

Toggle Object: This toggles the active state of the specified GameObject. On activation the GameObject is enabled, and on deactivation it's disabled.

Toggle Shader: This toggles the active state of a Screen Shader template instance, used for the multiple-material screen shader approach where each button activates a different shader. See [Screen Shader Setup].

Toggle Shader Keyword: This enables the specified shader keyword on the chosen renderer's material on activation, and disables it on deactivation. Most shader sets manage keywords automatically through their property toggles, so you usually don't need this action for the officially supported shader sets.

Toggle Shader Property: This sets a shader property on the specified renderer's material to the active value, and reverts it to the default value on deactivation. Supports Float, Color, and Vector property types. The "Also Set Effect Toggle" option auto-enables the effect's master toggle keyword, which is required for most screen shader sets. Use the Search button to pick a property from the assigned material.

Toggle Skybox: This swaps the scene’s skybox to the specified material on activation, and reverts to the scene’s starting skybox on deactivation. See the Skybox Folder template for an example.

Toggle Transform: This swaps the position, rotation, and/or scale of the specified Transform between an active and default state. Each axis can be enabled independently, and the action can target world or local space.

Toggle Udon Variable: This sets a public field on the specified UdonSharpBehaviour to the active value on activation, and to the default value on deactivation. Supports Float, Bool, Int, and String variable types. Use the Search button to pick a variable from the target behaviour.

Command Actions:

Apply Material: This applies the specified material to the renderer slot on press. Unlike Toggle Material, there is no off state — the swap is one-shot and persists until something else changes the material.

Apply Skybox: This swaps the scene’s skybox to the specified material on press. Like Apply Material, the swap is one-shot with no automatic restore.

Set Autochange Group State: This forces the specified auto-changing group on or off on press. Unlike Toggle Autochange Group, this always sets the state to a fixed value rather than flipping between on and off.

Set Object State: This forces a GameObject to a specific active state on press. Useful for “always-on” or “always-off” buttons.

Set Shader Keyword: This forces a shader keyword to a specific state on press. The target state determines whether the keyword is enabled or disabled.

Set Shader Property: This writes a value to a shader property on press. With “Use Step” enabled, the value increments by the configured Step Amount on each press, optionally wrapping or clamping at the configured min/max bounds — useful for cycling through discrete property values like the AudioLink band selectors in the included templates.

Set Transform: This sets a Transform’s position, rotation, and/or scale to a fixed value on press. Each axis can be enabled independently, and the action can target world or local space.

Set Udon Variable: This writes a fixed value to a public field on the specified UdonSharpBehaviour on press. Supports Float, Bool, Int, and String variable types. With “Use Step” enabled, the value steps each press like Set Shader Property.

Teleport Object: This teleports a GameObject to the specified destination Transform on press. Position and optionally rotation are copied from the destination.

Teleport Player: This teleports the local player to the specified destination Transform on press. Position and optionally rotation are applied to the player.

Trigger Udon Event: This calls a public method on the specified UdonSharpBehaviour on press. Use the Search button to pick from the target's available public methods.

Selection Actions:

Next Color: Within a Color Palette, this advances the pending color to the next palette entry on press. Pair with a Set Color button using the same Color Palette Name to apply the pending color to a target.

Next Variant: Within a Variant Group, this advances the pending variant to the next item on press. Pair with a Set Variant button using the same Variant Group Name to apply the pending variant.

Previous Color: Like Next Color, but moves backward through the palette.

Previous Variant: Like Next Variant, but moves backward through the variant list.

Set Color: Owns the color palette and applies the pending color to the target renderer on press. Configure the palette colors directly on this action.

Set Variant: Owns the variant list and applies the pending variant on press. Each variant maps to a different value on a configured shader property.

Preset Actions:
See [Using Presets].

Display Actions:

Display Autochange Group: Displays the current state of the named auto-changing group on the button label. Read-only — pressing does nothing.

Display Color Palette: Displays the currently applied color from a Color Palette as the button's tint. Pair with a Set Color action elsewhere using the same Color Palette Name.

Display Controller: Displays a value sourced from the controller itself, such as the current folder name. Read-only.

Display Shader Property: Reads a shader property from a material and displays its current value on the button label. Useful for showing the current value of a property driven by other buttons or faders.

Display Stat: Displays a VRChat world stat (visit count, favorite count, etc.) on the button label. Some metrics require the world's API URL to be configured. Read-only.

Display Udon Variable: Reads a public variable on a UdonSharpBehaviour and displays its current value on the button label. Use the Search button to pick the variable from the target.

Display Variant Group: Displays the currently applied variant name from a Variant Group on the button label. Pair with a Set Variant entry using the same Variant Group Name.

System Actions:

Display Folder Name: Displays the current folder's name on the button label. Read-only.

Display Page Number: Displays the current page number on the button label. Read-only.

Go To Fader Page: Navigates to the specified fader page on press. See [Using Faders].

Go To Folder: Navigates to the specified folder on press.

Go To Page: Navigates to the specified page index within the current folder on press.

Next Fader Page: Advances to the next fader page on press. See [Using Faders].

Next Folder: Advances to the next folder on press.

Next Page: Advances to the next page in the current folder on press.

Previous Fader Page: Goes back to the previous fader page on press.

Previous Folder: Goes back to the previous folder on press.

Previous Page: Goes back to the previous page in the current folder on press.

Reset: Resets all entry states to their on-by-default values, all step values to defaults, and all color/variant selections to defaults. Useful for a "reset all" button.

Set Fader Mode: Switches between hand-collider and VRC pickup control modes for fader manipulation. Useful for desktop users or anyone who prefers grabbing faders rather than sliding them with hand colliders.

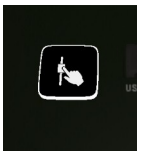
Set Whitelist: Forces the controller's whitelist on or off on press. Whitelist must be enabled in the EnigmaController inspector for this action to function. See [Whitelist (Access Control)].

Toggle Whitelist: Toggles the controller's whitelist between two states. The Default field is the value the whitelist returns to when the button deactivates; the active state writes the inverse. Whitelist must be enabled in the EnigmaController inspector for this action to function. See [Whitelist (Access Control)].

Using Faders

There are two types of faders within Enigma OS, static faders and dynamic faders. Think of each fader on the Mixer prefab as a "slot". Static faders are assigned to a slot permanently, while dynamic faders are only assigned to a slot when their associated button is enabled. Faders can control both float values and color values (hue shift), and can target shader properties (Material), Udon variables (Udon) or UI Slider components. You can also optionally target the scene skybox.

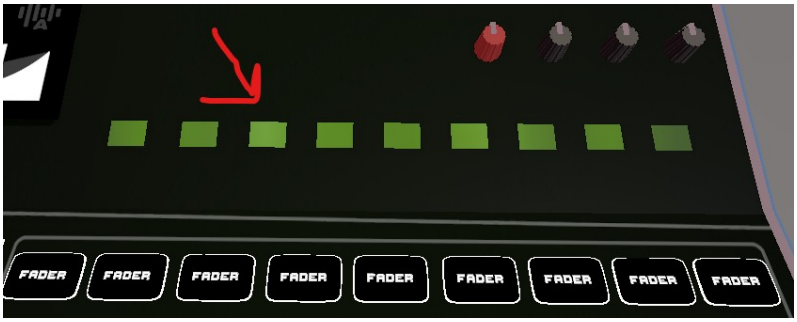
Faders can be controlled either by hand colliders (the tip of the index finger on each hand), or by normal VRC Pickup, allowing manipulation in the editor and in Desktop mode. The "Set Fader Mode" action controls this.



To assign a static fader, use the Faders foldout. The "+" button allows targeting multiple renderers or udon behaviors, letting a single fader control multiple VRSL lights for example.

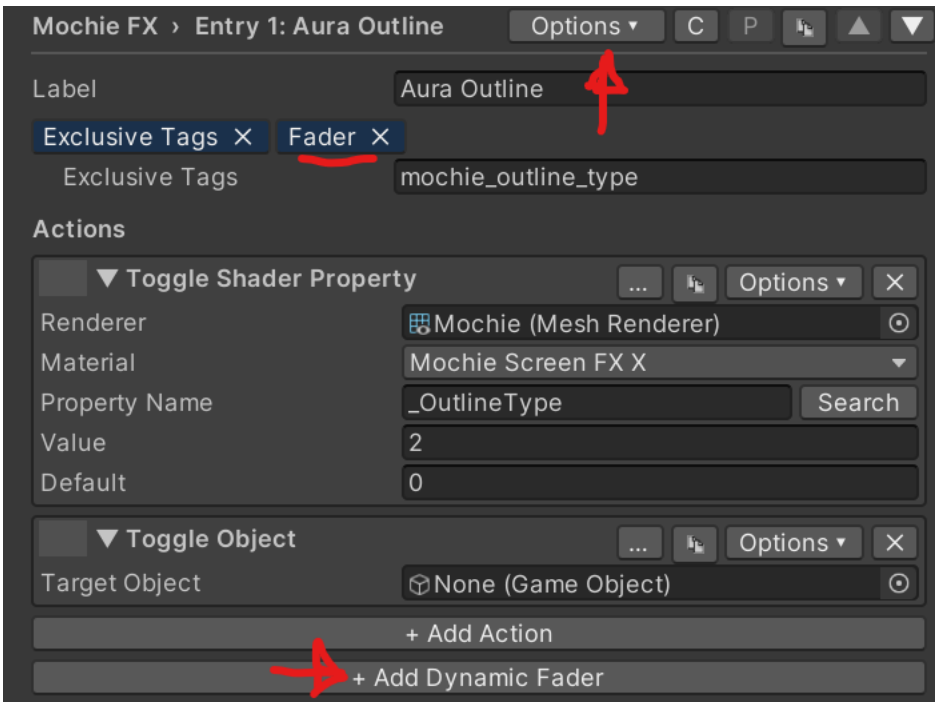
Min Value is the bottom position of the fader, while Max value is the top position of the fader. Max Value does not need to be greater than the Min Value. Default Value is the position the fader starts at when assigned.

If at any point you have more faders assigned than available slots, use the fader paging buttons to switch pages. Enigma OS includes several actions for fader navigation, so you can switch these to a previous / next fader page button if you prefer, letting you use the rest of the buttons as hotkeys for whatever actions you desire.

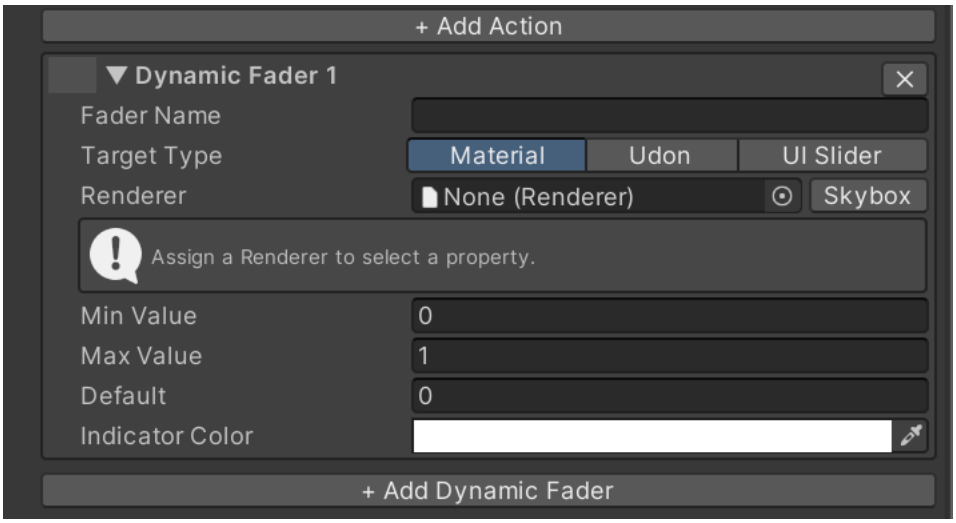


In the Options menu you can set “Always Visible” on a fader to pin that fader to stay visible across all fader pages. Keep in mind that this reduces the total number of assignable slots, if you set all 9 static faders to “Always Visible”, there will be no remaining slots for dynamic faders to be assigned.

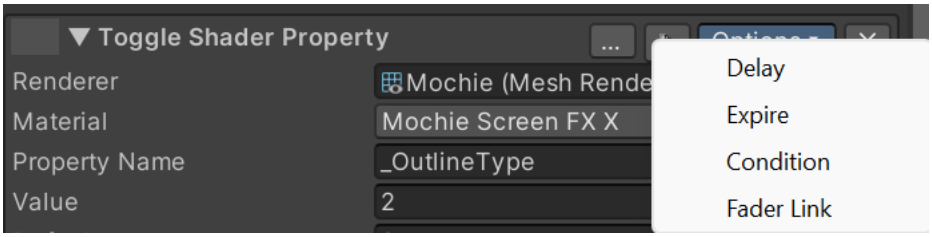
To configure dynamic faders, enable “Assign Fader When Active” in the Options menu of a button. You should see the “Fader” tag appear, and a new button “+ Add Dynamic Fader”.



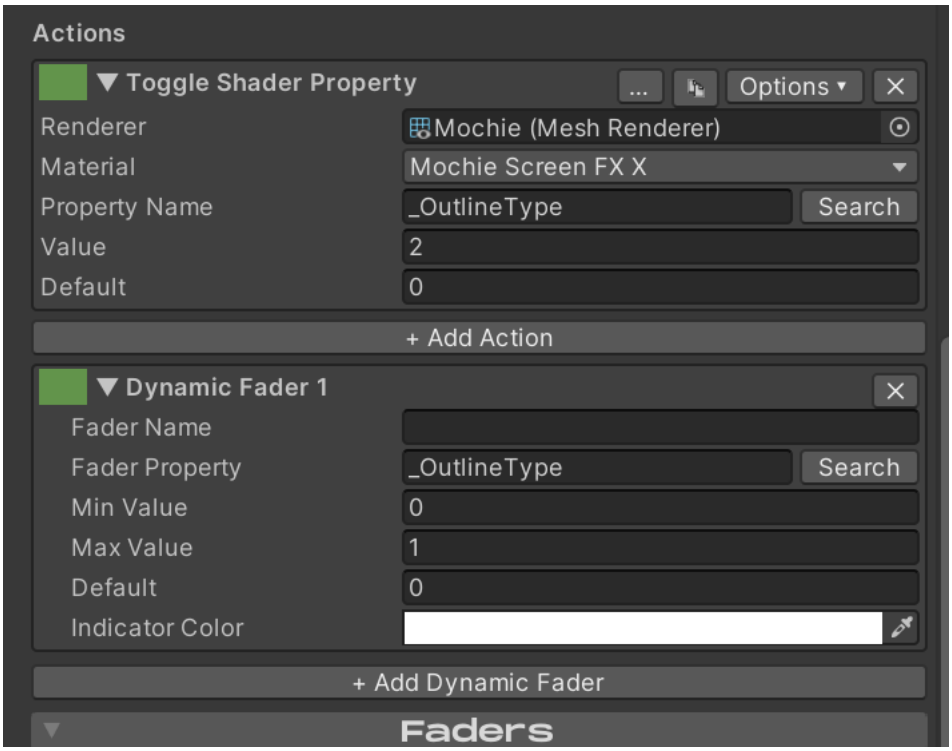
Clicking that button makes a dynamic fader that gets assigned when that button is in the “on” state. A button can assign multiple dynamic faders.



You can also make a dynamic fader by clicking “Options” on an action within a button and clicking “Fader Link”.



The benefit to this is that the newly created dynamic fader will inherit the renderer set in the action, saving time. If you change the renderer in the action, it will also change it in the linked fader. You can tell which faders are linked to which action as they will share a unique color in the drag box (green in the example below).



Dynamic faders will remember their positions when re-assigned. So if you disable a button and then enable it again, the position will snap to the position it was before being deactivated, not the default position.

Static faders are always assigned to the first slots on the controller, then any active dynamic faders.

Targeting VRSL Lights

VRSL stage lights drive their color through a Udon variable, not a material property. The fixture's script writes _Emission to its meshes every frame, so a fader that targets the material directly gets overwritten right away. Use the Udon path instead.

In the Faders foldout, drop the VRSL fixture's root GameObject into the Behaviour field, set the Property Type to Color, and set the Udon Variable Name to lightColorTint. The "+" button lets you add more fixtures to the same fader to drive a whole rig.

The same setup works for the VRSL GI variants — no extra configuration needed for the GI bounce color to follow.

Using Presets

Presets allow you to "snapshot" the current state of the controller in-game, and save that as a preset. Clicking that preset then sets that state, allowing you to launch multiple effects with one press. Presets save to persistent player data, allowing you to use the presets across all instances of that world. Whatever set of presets are loaded on the controller show for all users, these are

not local. This lets you share presets with others, though if they click Save they will overwrite any previously saved presets with the currently loaded set. Each operator can save and load their own set of presets.

You can use the Presets template to quickly make a Presets folder with all of the necessary actions. It includes 24 preset slots by default.

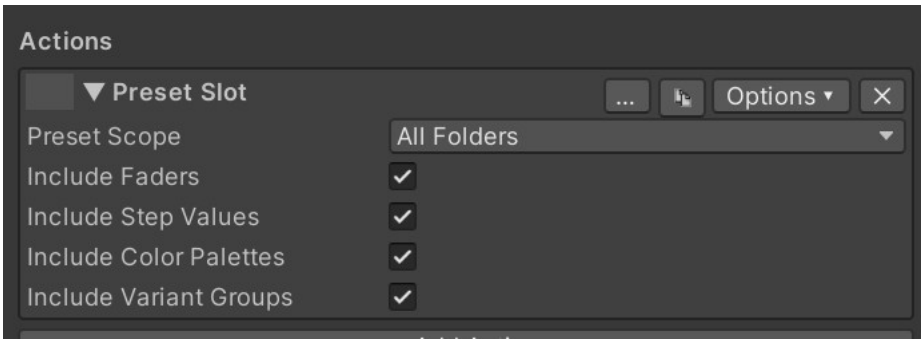
The available preset actions are:

Preset Slot: Represents a single preset slot. The Presets Folder template includes 24 of these. Press an empty slot to capture the controller's current state into it. Press a filled slot to recall its saved state. Press a slot while Clear Presets is active to clear that one slot. Slot data is network-synced across users in the world instance, but only persists across sessions when Save Presets is pressed (see below).

Clear Presets: This toggles a clear mode on the controller. While clear mode is active, the next Preset Slot press clears that one slot (clear mode then resets automatically). Press Clear Presets again to cancel without clearing. See [Using Presets].

Save Presets: This persists all populated preset slots to your VRChat PlayerData (per-user). Without pressing this, slot data only lives in the controller's runtime memory and is lost when you leave the world. Pressing Load Presets in a future session restores everything. See [Using Presets].

Load Presets: This restores all preset slots from your PlayerData, overwriting whatever's currently in the controller's slots. If the controller's layout has changed since the presets were saved, an "Incompatible Layout" warning appears on the button instead. See [Using Presets].



Within each Preset Slot action, you can configure which folders should be included in the snapshot, and optionally whether fader positions and color palette selections / variants are included. Buttons with preset actions are excluded from snapshotting.

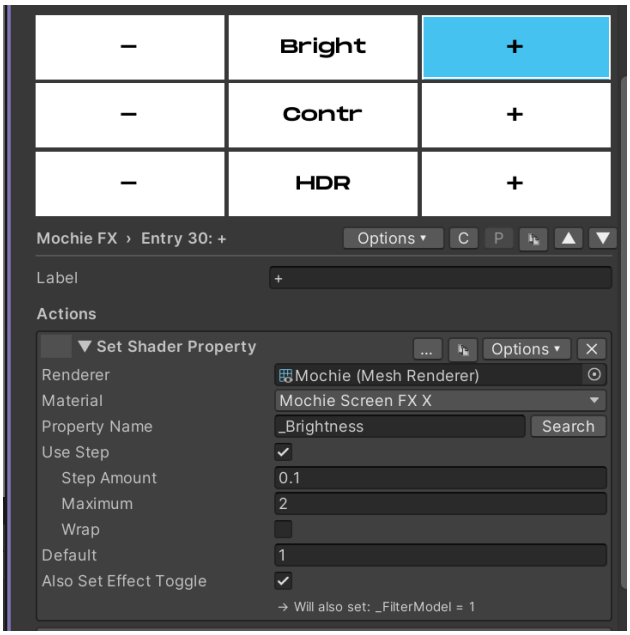
You should avoid using the “Load” button more than necessary, as this action syncs the large presets from your player data to ALL clients. This sync only happens once on load so it’s not that big of an issue, but it’s something to keep in mind.

Advanced Features (Step buttons, exclusivity, Custom Colors, Delays, Conditions, Color Palette, Autochanging)

Step Buttons

Step buttons allow you to increment a value by a specified amount within a specified range, optionally with wrapping. This allows one button to set many states instead of just one, useful for applications like an AudioLink band button, or - / + buttons. Examples of these are in the templates.

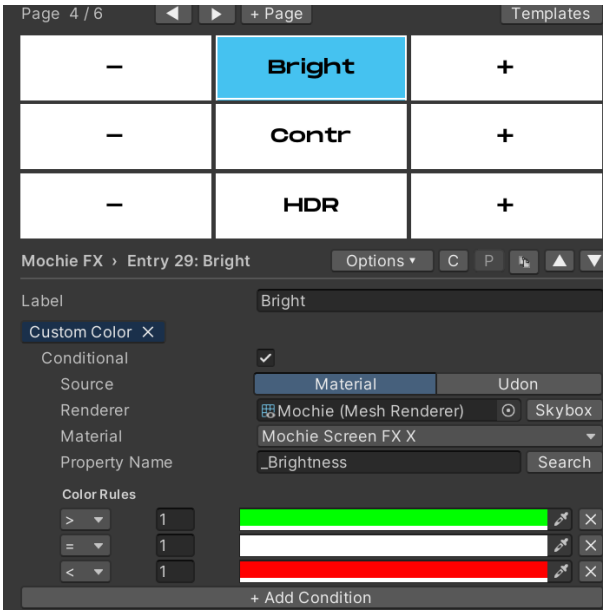
To make a step button, add either the “Set Shader Property” or “Set Udon Variable” action to a button, and then enable “Use Step”. In the below example, I’ve configured Brightness to increment by 0.1 on each press, up to a maximum of 2. The “-“ button has the same configuration, but with a step amount of -0.1.



The button in the middle uses a Display action and Custom Color to display the value and provide visual feedback for the step buttons. See [Display Actions and Custom Colors].

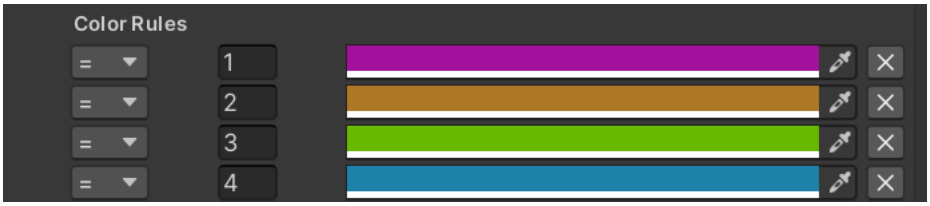
Custom Colors and Display Actions

The “Custom Color” in the Options menu of each button lets you choose what color the button indicator shows, instead of the button using the default colors in the Settings foldout. Optionally, you can also enable “Conditional” to have the color be controlled by the value of a shader property or udon variable.

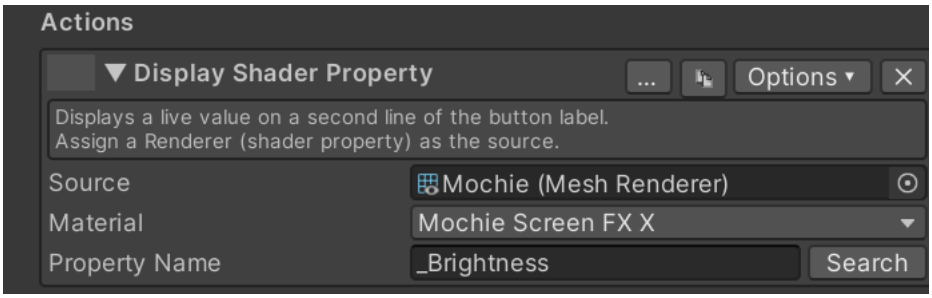


In this example, the button is white (default inactive color) when the value is 0, green when the value is positive, and red when the value is negative.

Using this same method, we can also use Custom Color for an AudioLink band button. Keep in mind that the bands are dependent on the shader (0-3 for Mochie FX, 1-4 for Bean FX)



Display actions display a value on the second line of the button that has the action. For example, I am using a “Display Shader Property” action to display the value that is controlled by the step buttons on either side of the “Bright” button.

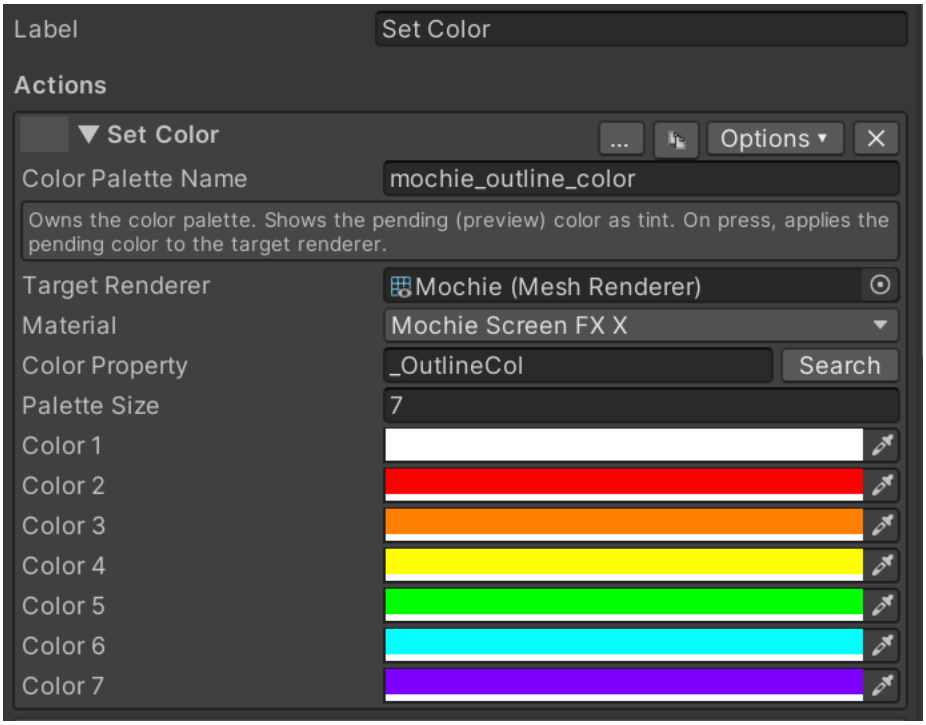


Color Palettes

Color Palettes allow you to define a set of colors, and then choose which color to apply. For example, in the Mochie FX template we accomplish this using three buttons: Cur Color, Set Color and Nxt Color.

The Set Color action defines the Color Palette. Here you define a name for the color palette and the colors in the palette. All three buttons for the palette use this same name, allowing you to make more than one color palette. The pending color is shown on this button, and on press it sets

the color on the target shader property.



The Next Color action lets you cycle through the colors, changing the pending color. This lets you choose what color you want to apply, then you press the Set Color button to apply it.

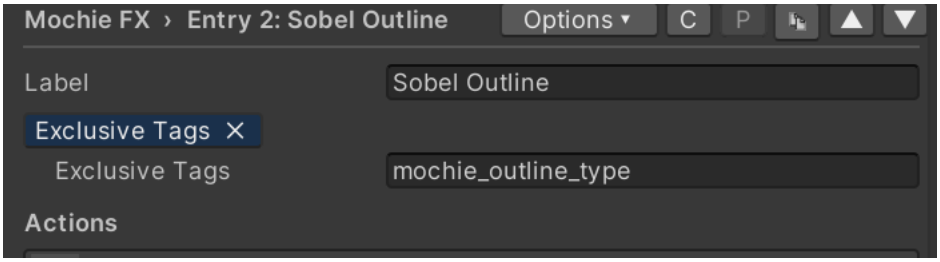
The Cur Color button in my example simply displays the active color. The active color is what the shader is set to, not the pending color on the Set Color button.

There is also a Previous Color action if desired.

Exclusivity

Exclusivity allows toggling on a button to disable other buttons with the same exclusive group. This system is inspired by and works similar to VRCFury exclusive tags. This is useful in a number of applications, such as effects that can not both be enabled at the same time.

An example of this is in the Mochie FX template for the Aura Outline and Sobel Outline buttons. To enable exclusivity, enable “Use Exclusive Tags” in the Options menu of the button. An “Exclusive Tags” tag should appear.



In this example, both Aura Outline and Sobel Outline share the tag “mochie_outline_type”. A button can belong to more than one exclusive group, enter exclusive tags comma delimited. Enabling one of these buttons then disables the other. Only one button in an exclusive group can be enabled at a time. However, all buttons in an exclusive group can be disabled.

If you want one button within a group to always be enabled, such as a group of toggle material action buttons, you can also enable “Exclusive Off” in the Options menu. Whichever button has this tag will be enabled when a button in the exclusive group is disabled. There is also a “Make Folder Exclusive” and “Clear Exclusivity” option to quickly set an entire folder to be exclusive, useful for applications like a Skybox folder.

Variant Groups

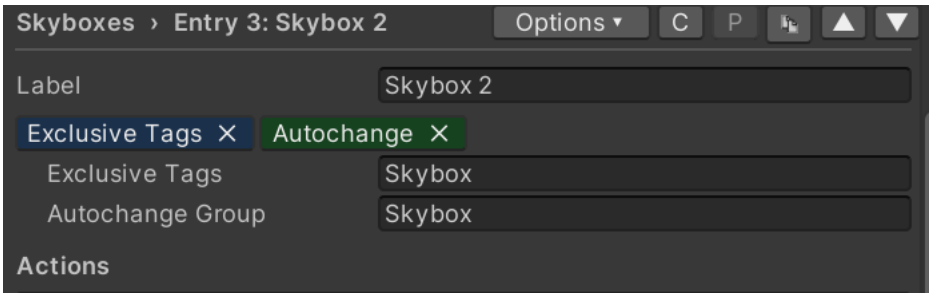
Variant Groups work similar to Color Palettes, but instead of defining a set of colors, you can define a set of values. This lets you configure a selector for a set of values and choose which to apply, instead of stepping through values with a step button. Additionally, variant groups support textures. This is useful for defining a set of textures for effects like overlays and triplanar effects, which is impossible to do through step actions.

See [Color Palettes] for setting this up, the setup is fundamentally the same.

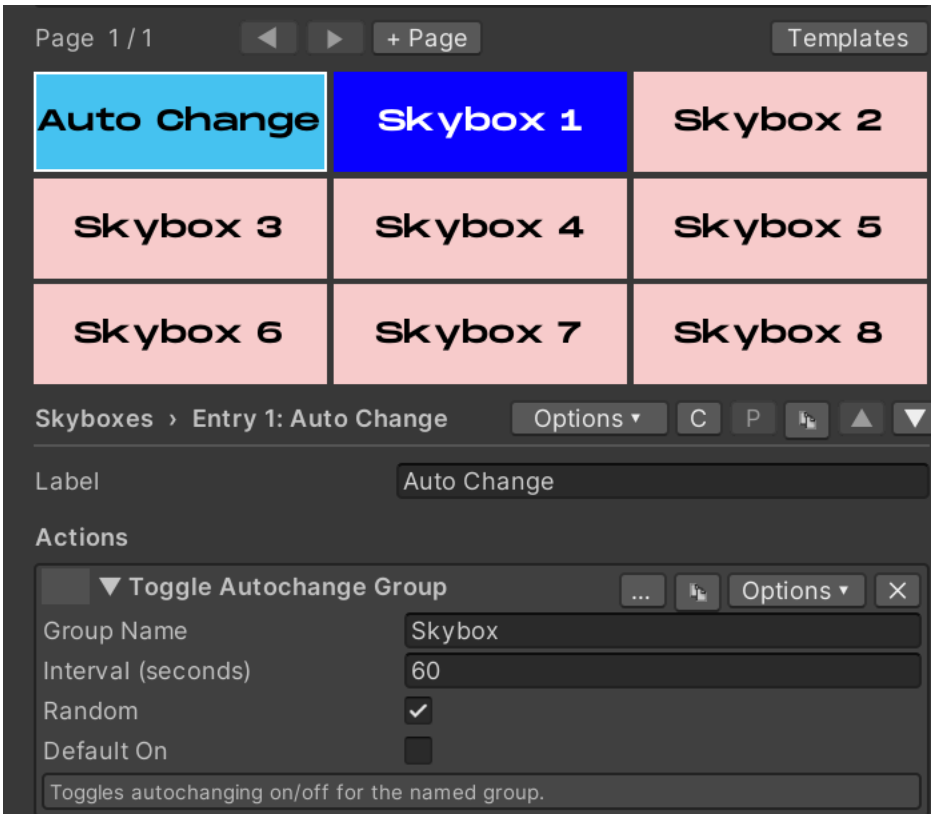
Variant Groups have not been extensively tested yet, so use at your own risk.

Auto-Changing

Auto-Changing allows Enigma OS to cycle between buttons on a specified interval. An example of this is in the Skybox template.



Enter a name for the Autochange group. All buttons you want to be autochanged should share this group name.

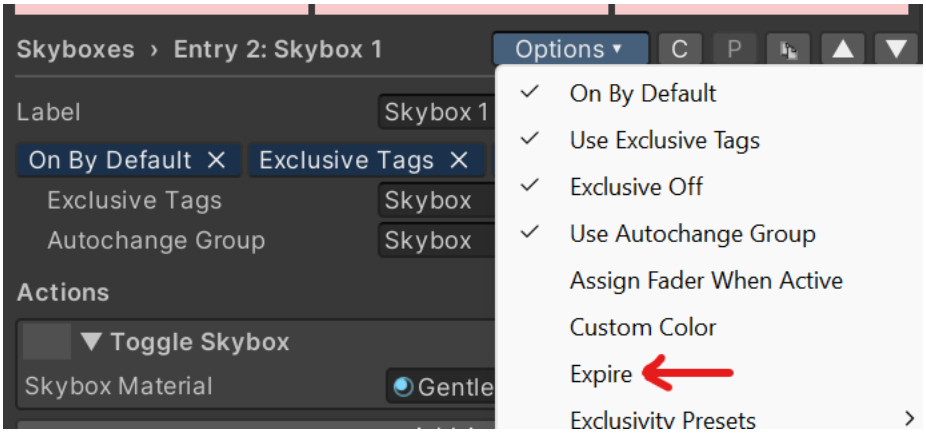


On a separate button, add a “Toggle Autochange Group” action. This action enables or disables auto-changing for the specified group. Set the desired change interval, and whether the auto-changing should be on by default when the controller is initialized. The “Random” checkbox controls if auto-changing picks a random button (besides the currently active one), or if it goes in order as configured in the controller (Skybox 1 -> Skybox 2 -> Skybox 3 for example).

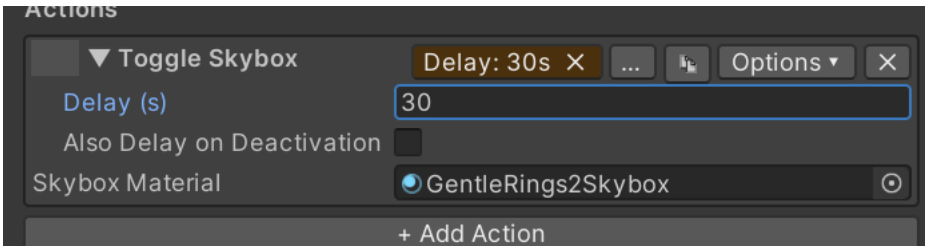
You can make any number of auto-change groups as you desire, just make sure you have separate toggle autochange buttons to control them. Unlike exclusive groups, a button can only be assigned to a single auto-change group.

Expire / Delay / Fade / Conditions

You can configure a button to expire after a certain amount of time. Enable “Expire” in the button’s Options menu. After activating the button, Enigma OS will wait that amount of time and then disable the button.

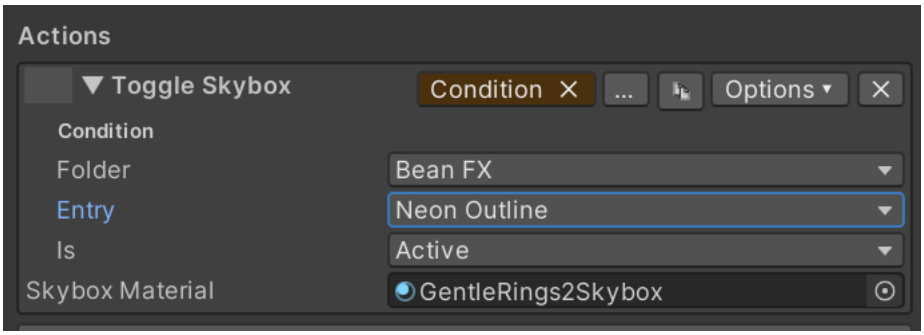


Actions can also be configured to have a delay before they are executed. Each action can have a separate delay, allowing staggered action execution. Optionally you can also have it delay deactivation as well. Enable the “Delay” option in the action’s Options menu and set the wait time in seconds.



Set Shader Property actions (Float, Color, or Vector properties) and Set Udon Variable actions (float and int variables) can fade smoothly to their value instead of snapping. Enable the “Fade” option in the action’s Options menu and set the fade time in seconds. On activation the property fades from its current value to the target value over that time — for example, a toggle setting an outline strength from 0 to 1 with a 3 second fade will fade the outline in over 3 seconds. By default, turning the button off snaps back to the default value immediately; enable “Also Fade on Deactivation” to fade back out over the same time instead (this checkbox only appears on Toggle actions — one-shot Set actions have no deactivation). Fade can be combined with Delay — the fade starts when the delayed action fires.

You can also set an action to only execute if a condition is true. This is really just a proof of concept right now. Currently, the only condition possible is the active state of another configured button on the controller. If you have a use case for other conditions, submit a suggestion in the Discord and I can add it.



Using Multiple Enigma OS controllers

Enigma OS supports using multiple controller prefabs in the same scene. If you're using multiple controllers for separate purposes, you do not need any additional setup.

If you want to use two or more controllers for the same purpose, for example screen shaders in separate areas of your world, or separate skyboxes in different areas, then you need to add the "Enigma Controller Boundary" script to a gameobject with a trigger collider, sized around the area that the controller should be active in. This script re-syncs the visual state when the local player enters that area and disables controllers in other areas so the player only receives actions from the controller in the area of the world they are currently in. You can optionally disable the disabling behavior.

Whitelist (Access Control)

Enigma OS supports whitelisting users through a manual username list or various third party integrations. Supply usernames in the Manual Username List (case insensitive). If desired, add a reference to a third party integration. If one is supplied, Enigma OS will use that whitelist instead, using the manual list ONLY in the case that it fails to pull the whitelist from the third party whitelist, which would be unusual.

Enigma OS supports OhGeezCmon Access Control, ProTV Managed Whitelists, and Flatline Sync. The download links for these are in [Installation and Dependencies]. You can assign more than one; see [Using more than one whitelist together] below for how Enigma OS keeps them in sync.

The whitelist also gates the syncing of the hand collider objects for fader control.

Using more than one whitelist together

When you assign more than one integration, Enigma OS keeps them in sync by mirroring changes downward from the highest-priority system you have assigned. In priority order, highest first, that is:

1. OhGeezCmon Access Control
2. ProTV Managed Whitelist
3. Flatline Sync
4. Manual Username List

The highest assigned system is the source of truth, so **make your whitelist edits there**. Enigma OS copies those changes down into the other systems automatically. The lower systems act as read-only mirrors: a change made directly in a lower system (for example, authorizing someone in Flatline while OhGeezCmon is also assigned) does not travel back up, and may be overwritten the next time the source of truth syncs.

Pre-authorized users propagate too. Names you seed ahead of time (such as OhGeezCmon's "Players With Starting Access") are pushed into the other systems, so those players are already authorized the moment they join, even if they were not in the world when you added them.

Enigma OS checks for changes every couple of seconds, so an update can take a moment to appear in the other systems. That short delay is normal.

Setup requirements

- **ProTV Managed Whitelist:** the Managed Whitelist must have its TV reference wired to your ProTV TV. Enigma OS will not push into a Managed Whitelist that is not connected to a TV.
- **Flatline Sync:** turn on Flatline's Use External Whitelist option. Enigma OS drives Flatline's admin menu directly, and this prevents Flatline's own startup whitelist logic from conflicting with it.
- **ProTV panels and inactive or whitelist-gated containers:** do not nest ProTV UI panels (such as MediaControls) under any GameObject that is inactive on world start or whose visibility is gated by an OhGeezCmon whitelist. A ProTV panel that is inactive while the TV starts up can come back with a blank track title and time display. Keep media-control panels somewhere that is active at world load, and use the whitelist to gate interaction rather than visibility. Enigma OS also protects against this: it never hides the admin menu before it has granted it once in a session, and when it re-shows the menu it asks ProTV to refresh its panels.

ProTV super users and what happens when the host leaves

ProTV only lets a ProTV super user edit its whitelist. ProTV treats the following as super users:

- The instance owner (the person who created the instance), while they are present.
- Anyone listed in the Managed Whitelist's Super Users field.
- The first instance master, if you have enabled ProTV's first-master-is-super options.

Importantly, the **instance master is not automatically a super user**. If the instance owner (or whoever was acting as your super user) leaves, the master role passes to another player, but that new master cannot edit the ProTV whitelist. When this happens, Enigma OS can still update OhGeezCmon, Flatline, and its own access; only the ProTV list stops receiving new changes until a super user is present again. Anyone already on the ProTV list stays authorized.

To keep the ProTV whitelist syncing even after you leave, add your trusted co-hosts to the Managed Whitelist's Super Users field before uploading. As long as at least one super user is in the instance, changes keep flowing into ProTV.

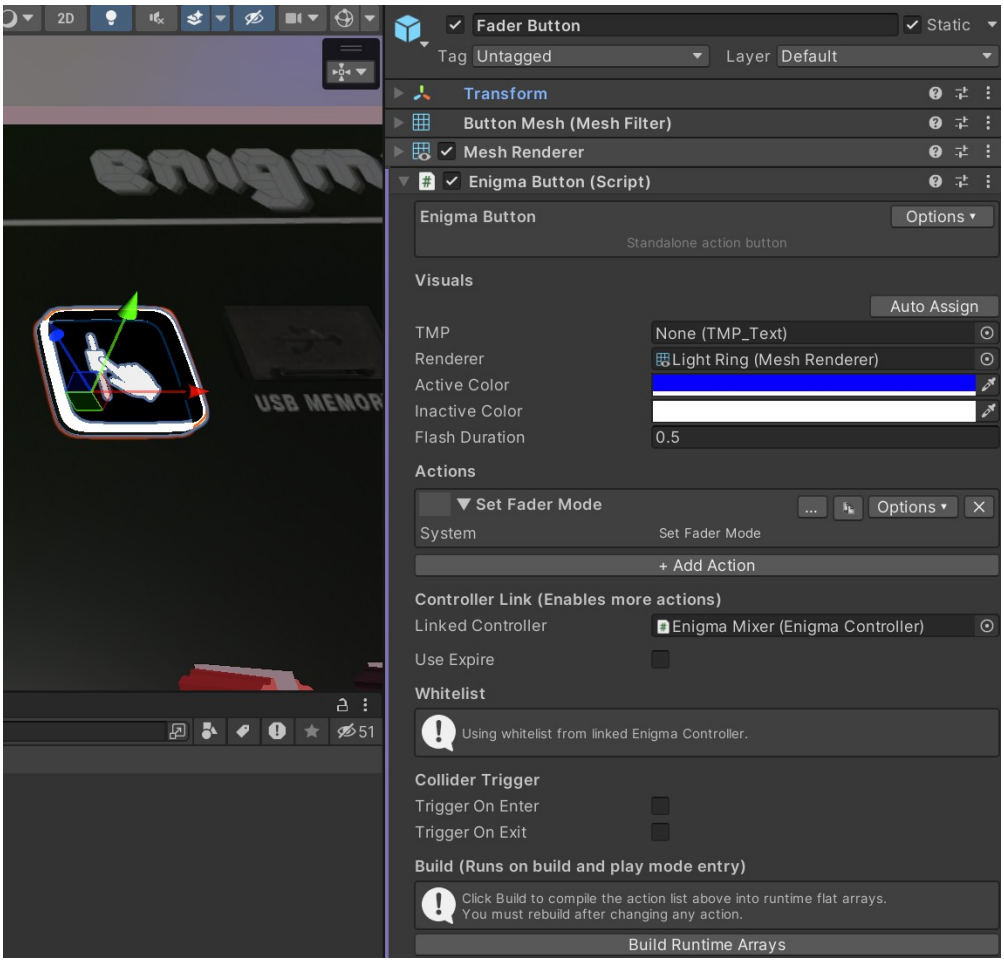
Note: OhGeezCmon Access Control automatically grants admin access to whoever is the current instance master. This means the master always has full control of the whitelist (and therefore of Enigma OS). If that is not what you want, consider whether OhGeezCmon should be your top-priority system.

Standalone Buttons

You can also use Enigma OS as an independent action executor, standalone of the controller system. To do this, add the Enigma Button script to any gameobject with a trigger collider. This is useful for making toggles in your world independent of the controller. Enigma Button also

supports triggers from a collider, so you can trigger actions when a player enters a certain area of your world. Optionally, you can link a controller to inherit the controller's whitelist or execute actions that rely on the system (like System and Preset actions).

On the Enigma OS prefabs, the page navigation, folder navigation, page display and folder display, fader control button, fader paging hotkeys and reset button are configured using Enigma Button components. To edit these, select the button in the scene.



Making Custom Controllers

You are more than welcome to design your own controllers or modify the included prefabs!

Enigma OS supports any number of buttons / faders. Assign these in the Hardware foldout. You can click "Auto-Assign Buttons & Faders" to quickly assign all buttons/faders nested under the Buttons and Faders child objects. You should assign these in order as they are physically in the scene, top-to-bottom, left-to-right per row.

You can adjust the Button Grid in the Enigma OS editor to match the physical layout you create by using the Layout dropdown in the Settings foldout. In this example, I am using 12 buttons with 3 rows of 4 buttons.